Lecture-2

Data Link Layer (LLC and MAC)

Dr. Md. Imdadul Islam
Professor, Department of Computer Science and Engineering
Jahangirnagar University
https://www.juniv.edu/teachers/imdad

The function of logical link control layer (LLC) are: framing, flow control and error control.

Framing

✓Framing refers to the process of partitioning bit stream into discrete units or blocks of data called frame. Framing enables sending and receiving machines to synchronize the transmission and reception of data because frames have detectable boundaries.

✓One common framing procedure involves inserting flag characters before and after the transmitting data message.

Fig.1 Frame format

- ✓ The header of a frame, normally carries the source and destination addresses and other control information, and the trailer, which carries error detection or error correction redundant bits.

Fixed-Size Framing

Frames can be of fixed or variable size. In fixed-size framing, there is no need for defining the boundaries of the frames; the size itself can be used as a delimiter (indication or symbol of boundary). An example of this type of framing is the ATM, which uses frames of fixed size called cells.

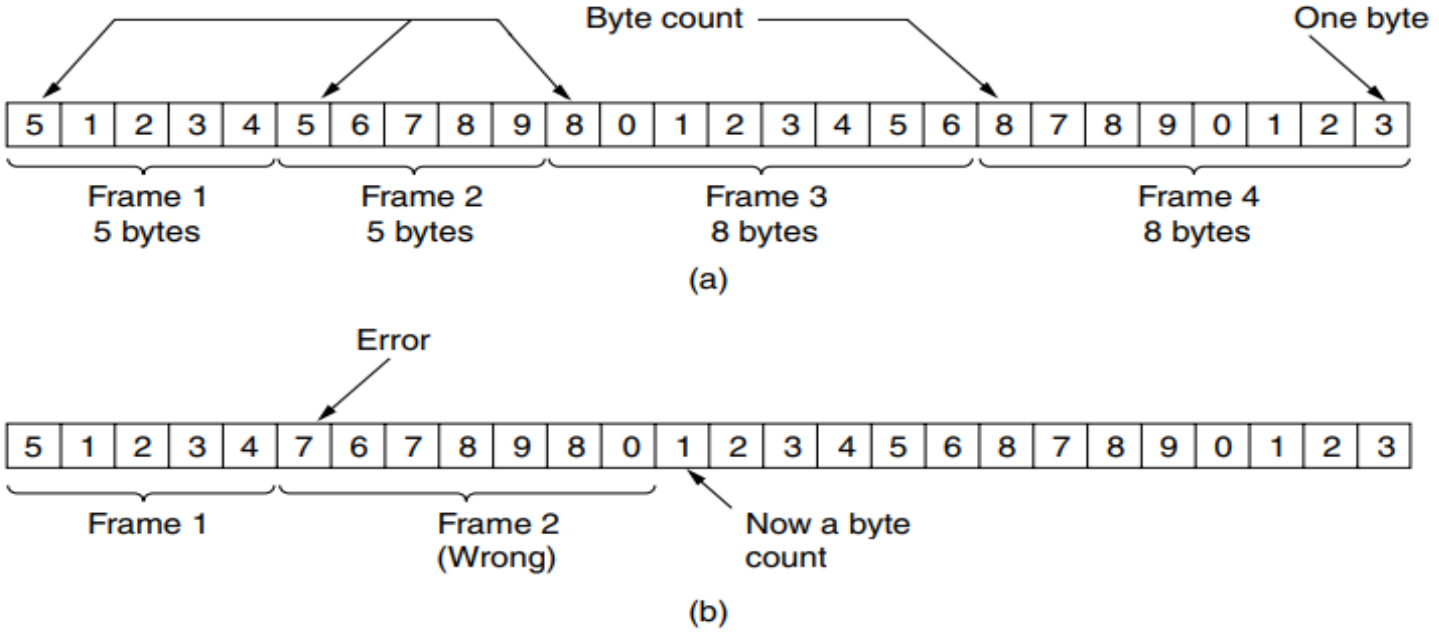
Variable-Size Framing

Variable-size framing is prevalent in local area networks. In variable-size framing, we need a way to define the end of the frame and the beginning of the next. Three approaches were used for this purpose:

- **Byte count method**
- **Character-oriented approach (flag byte with byte stuffing)**
- **bit-oriented approach (Flag bits with bit stuffing)**
- **Physical layer coding violations**

Byte count method

✓The first framing method (**Byte count method**) uses a field in the header to specify the number of characters in the frame. When the data link layer at the destination sees the character count, it knows how many characters follow and hence where the end of the frame is. This technique is shown in Fig. a below for four frames of sizes 5, 5, 8, and 8 characters, respectively.



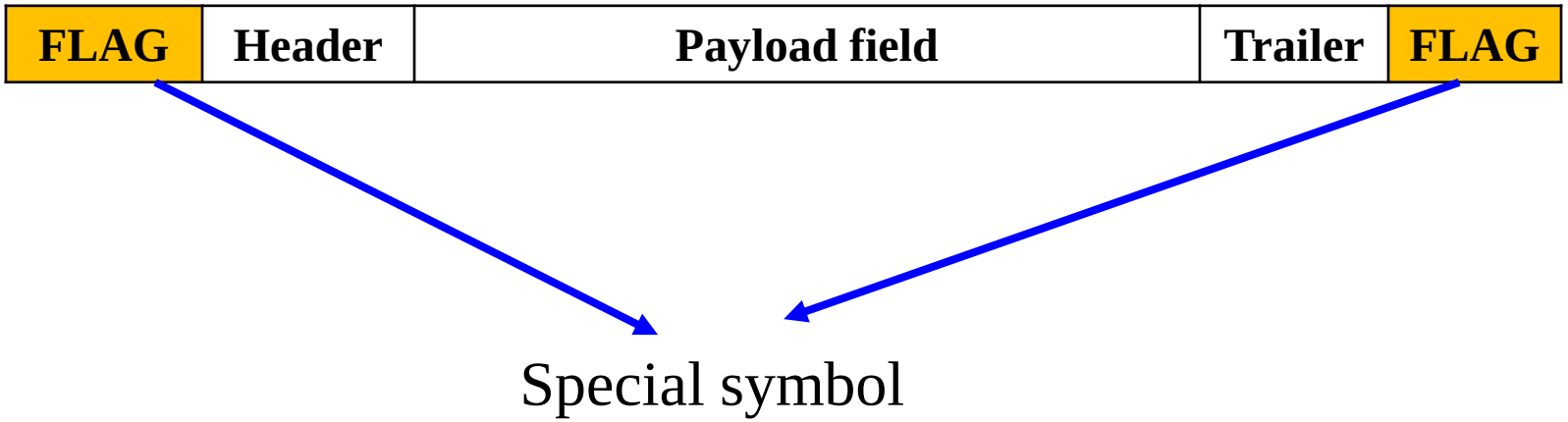
A character stream. (a) Without errors. (b) With one error.

✓The trouble with this algorithm is that the count can be garbled by a transmission error. For example, if the character count of 5 in the second frame of fig. b becomes a 7, the destination will get out of synchronization and will be unable to locate the start of the next frame

Character-Oriented Protocols (flag byte with byte stuffing)

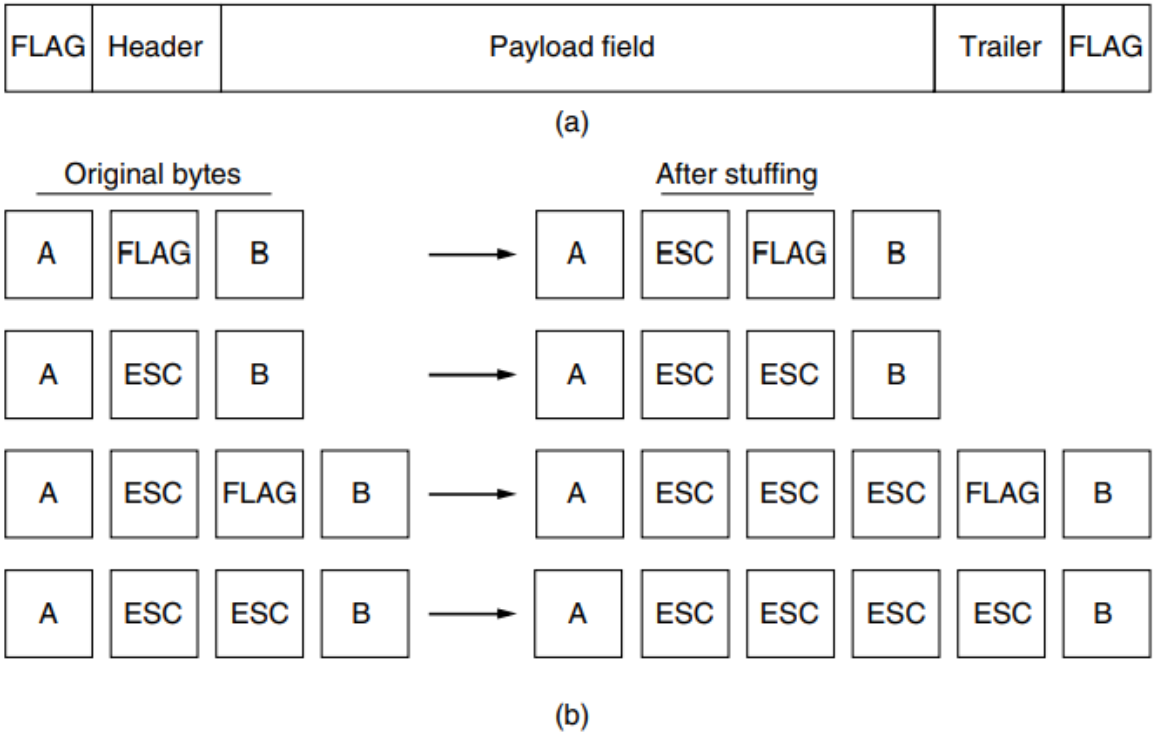
✓In **flag byte with byte stuffing**, a flag byte is used at both the starting and ending delimiter, as shown in Fig. below.

✓In this way, if the receiver ever loses synchronization, it can just search for the flag byte to find the end of the current frame. Two consecutive flag bytes indicate the end of one frame and start of the next one.



✓It may easily happen that the flag byte's bit pattern occurs in the data. This situation will usually interfere with the framing. One way to solve this problem is to have the sender's data link layer insert a special escape byte (ESC) just before each 'accidental' flag byte in the data.

✓The data link layer on the receiving end removes the escape byte before the data are given to the network layer. This technique is called byte stuffing or character stuffing. Thus, a framing flag byte can be distinguished from one in the data by the absence or presence of an escape byte before it.



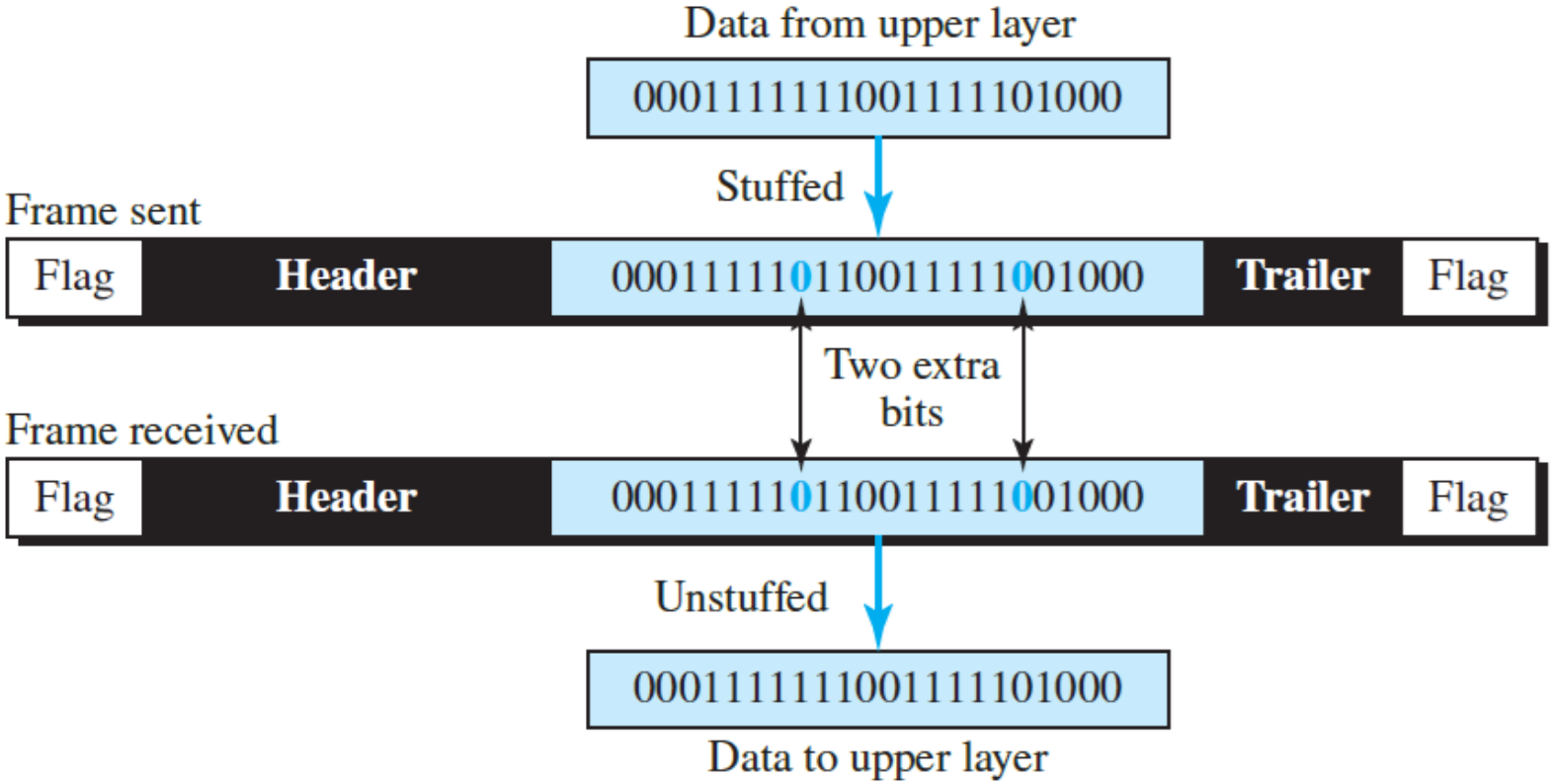
The major **disadvantage** of using this framing method is that it is closely tied to the use of 8-bit characters or byte stuffing. Not all character codes use 8-bit characters for example UNICODE uses 16-bit or 32 bit characters, pixel of each colored image is 24 bits will conflict with 8-bit characters.

Flag bits with bit stuffing

✓Each frame begins and ends with a special bit pattern, 01111110 (in fact, a flag byte). Whenever the sender encounters five consecutive 1s in the data (payload), it automatically stuffs a 0 bit into the outgoing bit stream.

✓When the receiver sees five consecutive incoming 1 bits, followed by a 0 bit, it automatically de-stuffs (i.e., deletes) the 0 bit. If the user data contain the flag pattern, **01111110**, this flag is transmitted as 011111010 but stored in the receiver's memory as 01111110.

Fig. below gives an example of bit stuffing.



With bit stuffing, the boundary between two frames can be unambiguously recognized by the flag pattern. Thus, if the receiver loses track of where it is, all it has to do is scan the input for flag sequences, since they can only occur at frame boundaries and never within the data.

- ✓ With both bit and byte stuffing, a side effect is that the length of a frame now depends on the contents of the data it carries.
- ✓ For instance, if there are no flag bytes in the data, 100 bytes might be carried in a frame of roughly 100 bytes (best case).
- ✓ If, however, the data consists solely of flag bytes, each flag byte will be escaped and the frame will become roughly 200 bytes long (worst case). (byte stuffing case)
- ✓ With bit stuffing, the increase would be roughly 12.5% as 1 bit is added to every byte. (bit stuffing case)

Physical layer coding violations

The fourth method of framing is to use a shortcut from the physical layer, where pulse sequence of special line code (shape of pulse) which is not used inside data pulse of a frame can be used as the head and tail of the frame.

Error Control

The term error control refers to the process of guaranteeing reliable data delivery. Two basic strategies exist for dealing with error.

- The first method, *error correction through retransmission*, involves providing enough information in the data stream so the receiving node can detect an error. Once an error is detected the receiving node can then request the sender to retransmit that unit data.

- The second method is *autonomous error correction*, involves proving redundant information in the data stream so the destination node can both detect and correct any error autonomously.

❑The concept of error detection, acknowledgement, and retransmission are collectively referred to as *automatic repeat request* (ARQ). Again when the receiver corrects the frame called *forward error correction* (FEC).

Error Detection Coding

The Cyclic Redundancy Check (CRC)

- ✓ For CRC coding we need a basic mathematics of 'bit sequence to polynomial conversion'. For example, an 8-bit message consisting of the bits, $M = 1\ 0\ 0\ 1\ 1\ 0\ 1\ 0$ corresponds to the polynomial,

$$M(x) = 1\times x^7 + 0\times x^6 + 0\times x^5 + 1\times x^4 + 1\times x^3 + 0\times x^2 + 1\times x^1 + 0\times x^0$$
$$= x^7 + x^4 + x^3 + x$$

Example-1

$M = 1\ 0\ 0\ 1\ 0\ 0\ 1 \leftrightarrow M(x) = x^6 + x^3 + 1$

$$S(x) = a_0 + a_1 x + a_2 x^2 + \ldots \quad \ldots \quad \ldots + a_n x^n$$

Steps of determining transmitted polynomial

We wanted to create a polynomial $T(x)$ for transmission that is derived from the original message $M(x)$. Transmitted bit string T is r bits longer than M, and $T(x)$ is exactly divisible by $G(x)$, where r is the degree of $G(x)$.

We can derive $T(x)$ from $M(x)$ in the following way:

1. Multiply the message polynomial $M(x)$ by x^r, which is an equivalent polynomial of a sequence derived by adding r zeros at the end of the message.
2. Divide $x^rM(x)$ by $G(x)$ and find the remainder $R(x)$.

3. Subtract the remainder $R(x)$ from the dividend $x^rM(x)$ to get $T(x)$.

It is obvious that $T(x)$ is exactly divisible by $G(x)$.

Example-1:

Given, Message bit string M = 1 0 1 0 0 0 1 1 0 1 (10 bits)

Generator bit string G = 1 1 0 1 0 1 (6 bits)

Determine remainder polynomial $R(x)$ and transmitted polynomial $T(x)$.

Ans.

Given, Message bit string, M = 1 0 1 0 0 0 1 1 0 1 (10 bits)

Generator bit string, G = 1 1 0 1 0 1 (6 bits)

Determine $M(x)$ and $G(x)$.

$$\therefore M(x) = x^9 + x^7 + x^3 + x^2 + x^0 \qquad G(x) = x^5 + x^4 + x^2 + x^0$$

Degree of $G(x)$, r = 5.

Let us create another polynomial, $x^5 M(x) = x^{14} + x^{12} + x^8 + x^7 + x^5$

$x^5 + x^4 + x^2 + 1$ ⟌ $x^{14} + x^{12} + x^8 + x^7 + x^5$ ⟌ $x^9 + x^8 + x^6 + x^4 + x^2 + x$

$x^{14} + x^{13} + x^{11} + x^9$

$x^{13} + x^{12} + x^{11} + x^9 + x^8 + x^7 + x^5$

$x^{13} + x^{12} + x^{10} + x^8$

$x^{11} + x^{10} + x^9 + x^7 + x^5$

$x^{11} + x^{10} + x^8 + x^6$

$x^9 + x^8 + x^7 + x^6 + x^5$

$x^9 + x^8 + x^6 + x^4$

$x^7 + x^5 + x^4$

$x^7 + x^6 + x^4 + x^2$

$x^6 + x^5 + x^2$

$x^6 + x^5 + x^3 + x$

$x^3 + x^2 + x$

$G(x)$ | $x^5 M(x)$ | $Q(x)$

$R(x)$

12 | 79 | 6

72

7

Modulo 2 operation

$\therefore R(x) = x^3 + x^2 + x \leftrightarrow 0\ 1\ 1\ 1\ 0$

$\therefore T(x) = x^5 M(x) - R(x)$

$= x^{14} + x^{12} + x^8 + x^7 + x^5 + x^3 + x^2 + x$

Transmitted bit sequences and polynomial

$$T(x) = x^{14} + x^{12} + x^8 + x^7 + x^5 + x^3 + x^2 + x$$

$$T = \mathbf{1\ 0\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 1\ 0}$$

$$R(x) = x^3 + x^2 + x \leftrightarrow 0\ \ 1\ \ 1\ \ 1\ \ 0 = R$$

$$G(x) = x^5 + x^4 + x^2 + x^0$$

$$\frac{T(x)}{G(x)} = Q(x) = x^9 + x^8 + x^6 + x^4 + x^2 + x$$

The received polynomial

$$R_e(x) = T(x) + E(x)$$

$M = 1\ 0\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1$

Error Polynomial

Case-I

$$T = \mathbf{1\ 0\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 1\ 1\ 0}$$

$$T(x) = x^{14} + x^{12} + x^{8} + x^{7} + x^{5} + x^{3} + x^{2} + x$$

If the received sequence is:

$$Re = \mathbf{1\ 0\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 1\ 1\ 0}$$

$$\mathrm{Re}(x) = x^{14} + x^{12} + x^{10} + x^{8} + x^{7} + x^{5} + x^{3} + x^{2} + x$$

$$= T(x) + E(x)$$

Where error polynomial, $E(x) = x^{10}$

Error Polynomial

Case-II

$$T = \mathbf{1\ 0\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 1\ 1\ 0}$$

$$T(x) = x^{14} + x^{12} + x^8 + x^7 + x^5 + x^3 + x^2 + x$$

If the received sequence is:

$$Re = \mathbf{1\ 0\ 1\ 0\ 0\ 0\ 0\ 1\ 0\ 1\ 0\ 1\ 1\ 1\ 0}$$

$$\mathrm{Re}(x) = x^{14} + x^{12} + x^7 + x^5 + x^3 + x^2 + x$$

$$= T(x) + E(x)$$

Where error polynomial, $E(x) = x^8$

Error Polynomial

Case-III

$$T = \mathbf{1\ 0\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 1\ 1\ 0}$$

$$T(x) = x^{14} + x^{12} + x^8 + x^7 + x^5 + x^3 + x^2 + x$$

If the received sequence is:

$$Re = \mathbf{1\ 0\ 0\ 0\ 1\ 0\ 1\ 1\ 0\ 1\ 0\ 0\ 1\ 1\ 0}$$

$$\mathrm{Re}(x) = x^{14} + x^{10} + x^8 + x^7 + x^5 + x^2 + x$$

$$= T(x) + E(x)$$

Where error polynomial, $E(x) = x^{12} + x^{10} + x^3$

If there is some error at detecting end then polynomial of the received string will be, $T(x) + E(x)$ where $E(x)$ is arises from error.

Now

$$\frac{R_e(x)}{G(x)} = \frac{T(x) + E(x)}{G(x)} = \frac{T(x)}{G(x)} + \frac{E(x)}{G(x)}$$

$$= Q(x) + \frac{E(x)}{G(x)}$$

Choice of $G(x)$:

Here if $E(x)$ is divisible by $G(x)$ then error can't be detected. To combat the situation, choosing of $G(x)$ has some criteria like below.

1. One common type of error is a single-bit error, which can be expressed as $E(x) = x^i$ when it affects bit position i. If we select $G(x)$ such that the first and the last term are nonzero, then we already have a two-term polynomial that cannot divide evenly into the one term $E(x)$.

2. For two bit error, $E(x) = x^m + x^r;\ m > r$

$= x^r(x^{m-r} + 1) = x^r(x^k + 1)$

In this case $G(x)$ has to be chosen such that it does not divide $x^k + 1$. For example if $G(x) = x^{15} + x^{14} + 1$ then $G(x)$ is unable to divide $x^k + 1$ for $k \leq 32{,}768$.

3. For odd number of bits in error then $E(x)$ will contains odd number of terms. For 3 bit error, $E(x) = x^5 + x^2 + 1$. If any polynomial has odd number of terms then it does not have any factor like $(x+1)$. If $G(x)$ has a factor of $(x+1)$ it would be unable to divide $E(x)$.

4. For burst error of length $k \le r$ then

$$E(x) = x^{k+i-1} + x^{k+i-2} + ... + x^{i+1} + x^{i}$$
$$= x^{i}(x^{k-1} + x^{k-2} + ... + 1)$$

If the degree of $G(x)$ is r and $r \ge k$-1 then $G(x)$ is simply unable to divide $E(x)$.

5. If the error sequence and generator are identical then the CRC code will failed to detect error. The probability of such case,

$$P = \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} ... (r+1) \text{ th term}$$

$= (1/2)^{r+1,}$ which is very small

$E(x) = G(x)$

G = 1 0 0 1 1 0 1 0 0 1
E = 1 0 0 1 1 0 1 0 0 1

Q. An information source generates message bit string, M = 1 0 0 0 1 0 1 1 0 1 0 1 (12 bits)
Generator bit string, G = 1 0 0 1 0 1 1 (7 bits)
i) Determine the polynomials: $R(x)$ and $T(x)$.
ii) How to avoid ambiguity of single bit, 2 bits and odd number of bit error, burst error and the problem of E(x)= G(x)?

Flow Control

- ✓ The various stations in a network may operate at **different speeds**. One of the tasks of the data link layer is to ensure that slow devices are not swamped with data from fast devices.

- ✓ The sending station must not send frames at a rate faster than the receiving station can absorb them.

- ✓ Flow control refers to the regulating of the rate of data flow from one device to another so that the receiver has enough time to consume the data in its receive buffer, before it overflows.

Stop-and-Wait Flow Control

- ✓ The simplest form of flow control, known as stop-and-wait flow control.
- ✓ A source entity transmits a frame. After the destination entity receives the frame, it indicates its willingness to accept another frame by sending back an **acknowledgment** to the frame just received.
- ✓ The source must wait until it receives the **acknowledgment** before sending the next frame.
- ✓ The destination can thus stop the flow of data simply by withholding acknowledgment.

- ✓ This protocol works fine when a message is sent in a few large frames but its performance is very poor for small frames.
- ✓ However, it is often needed by a source to break up a large block of data into smaller blocks to accommodate the receiver's limited buffer.
- ✓ Small frame sizes also facilitate faster error detection/correction and reduce the amount of data requires retransmission in the event of error detection.
- ✓ It is also necessary to reduce network congestion.
- ✓ For this case of small frame the line is always **underutilized**.

Sliding Window Flow Control

✓An enhancement to the stop-wait protocol is the sliding window concept, which improves data flow by having the receiver inform the sender of its available buffer space.

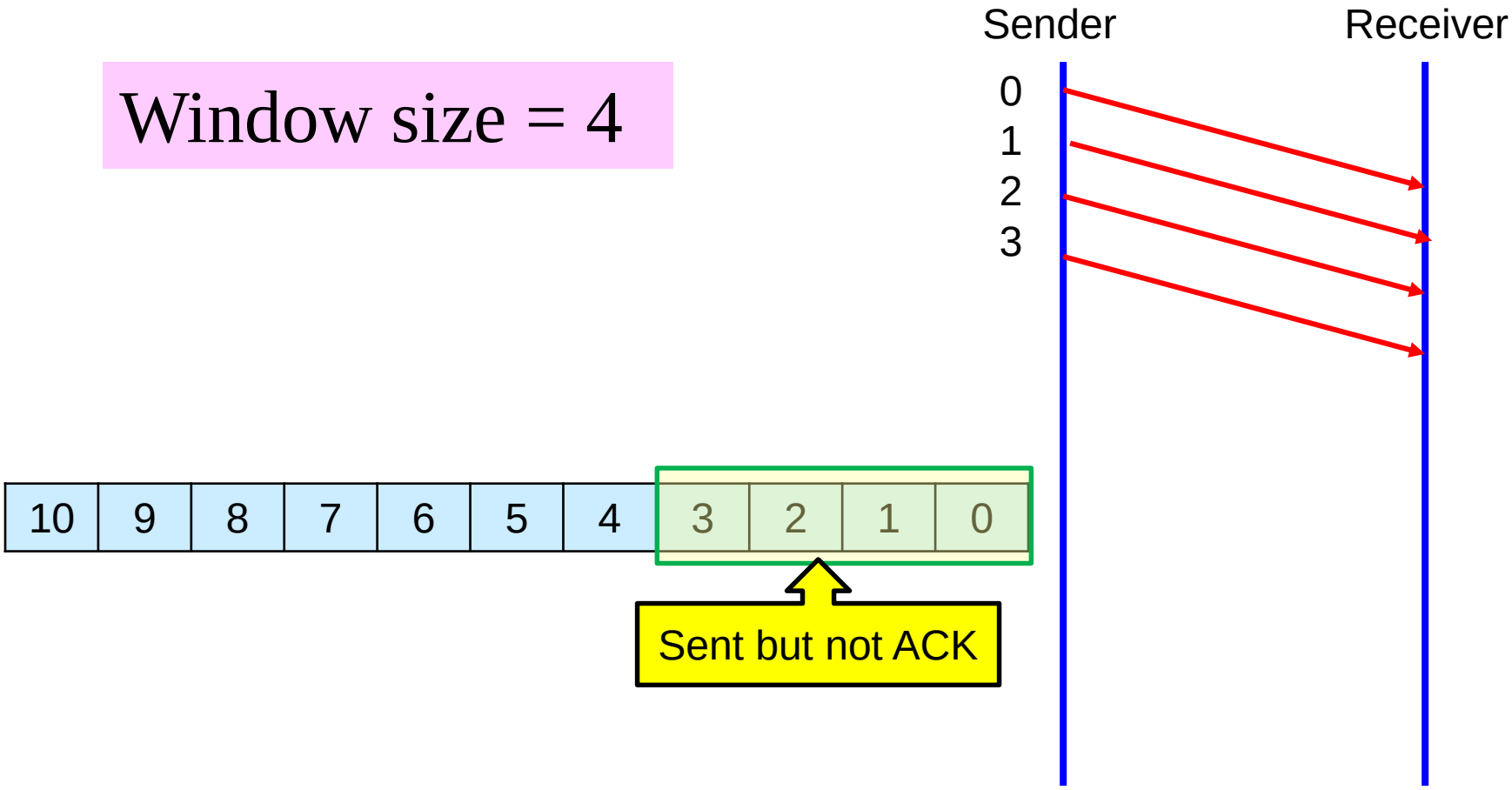
✓Doing so it enables the sender to transmit several frames without having to wait for acknowledgements to these frames (called size of window) as long as the number of frames sent does not overflow the receiver's buffers.

✓The sliding window concept is implemented by requiring the sender to sequentially number each data frame it sends to keep the track of frames.

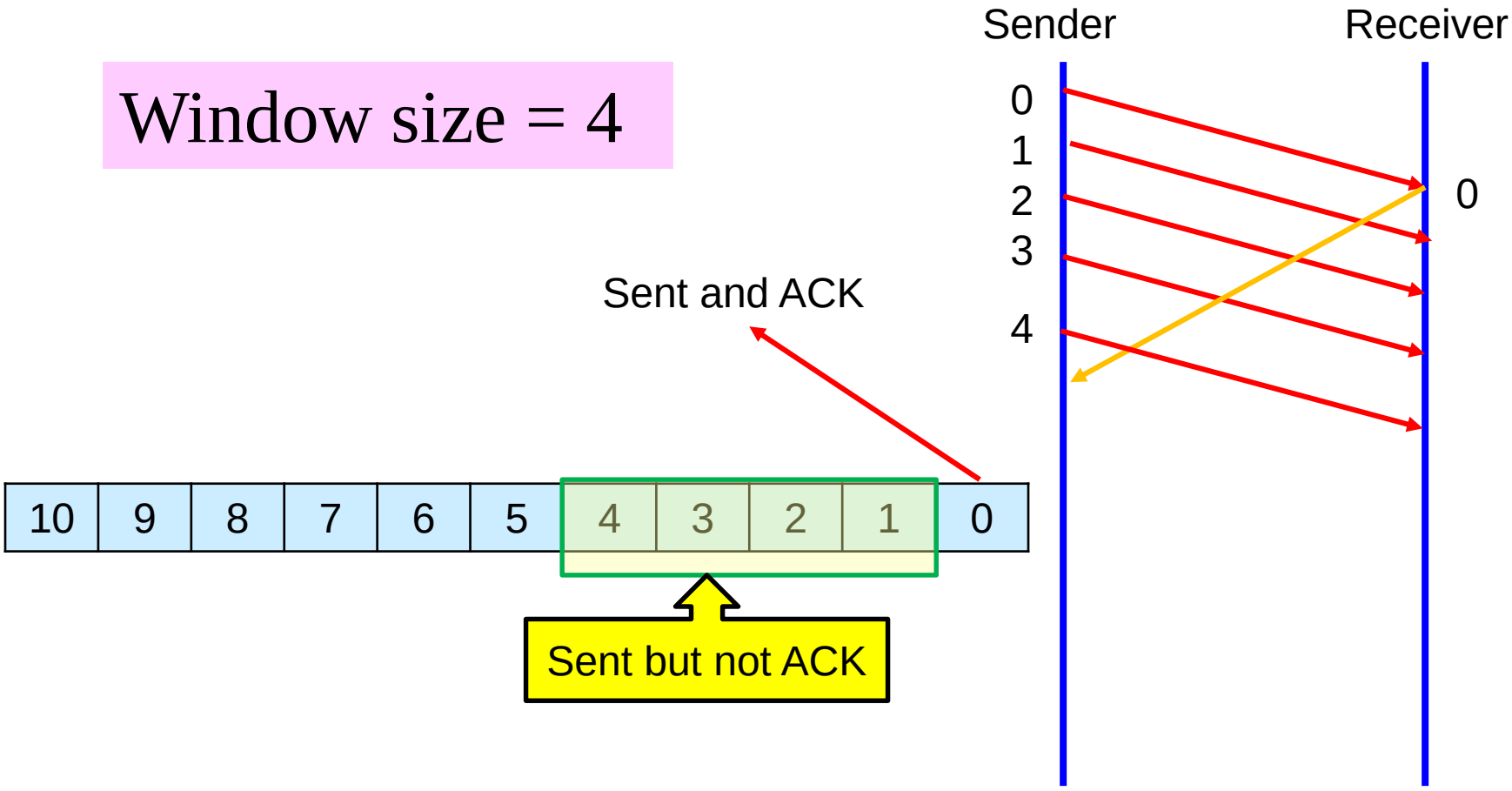
✓Flow-control protocols based on this concept are called **sliding window** protocol.

Consider a situation of fixed window size of 4. Every frame is numbered like 0, 1, 2, …10.

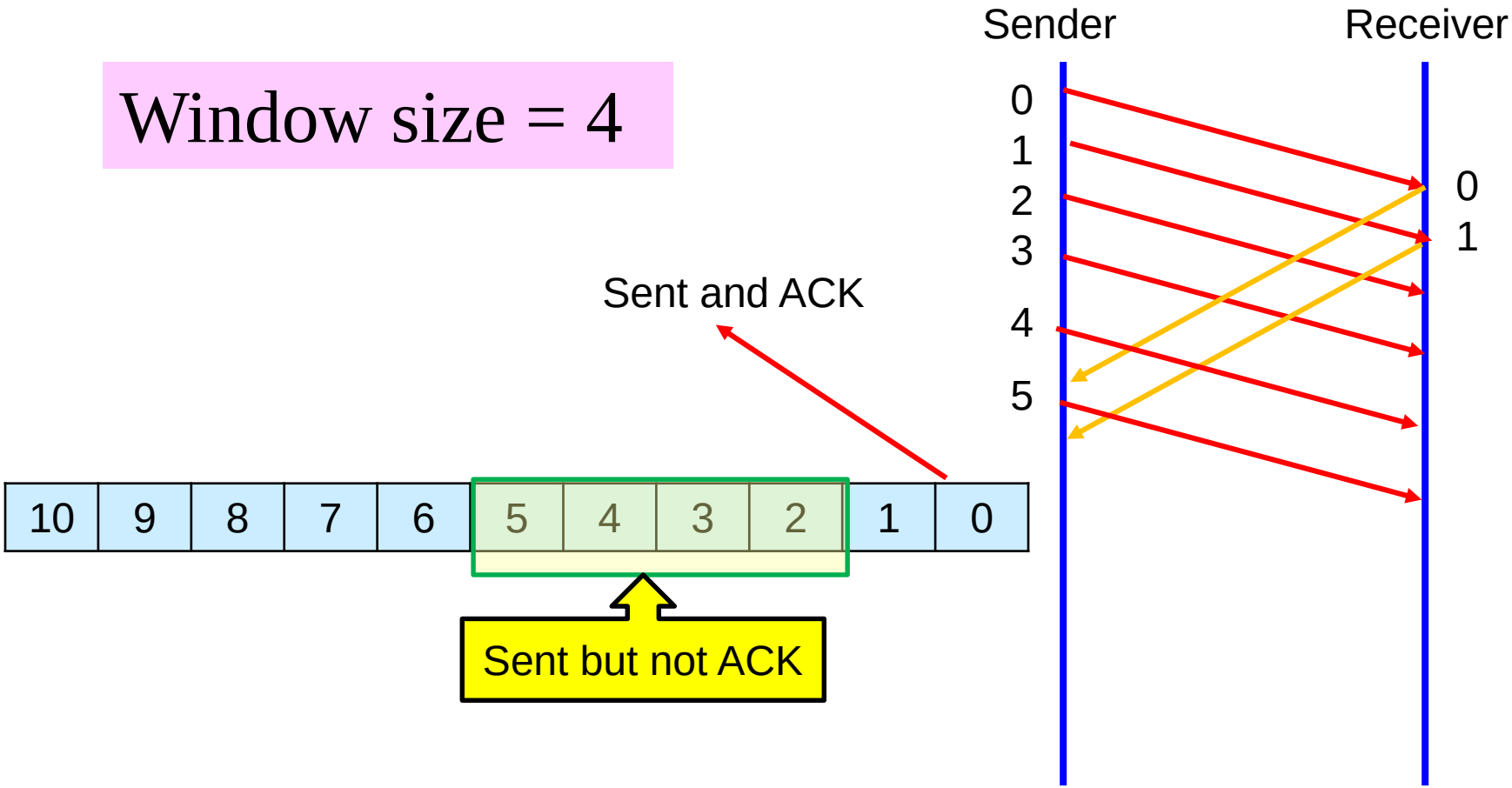
Window size = 4



Window size = 4



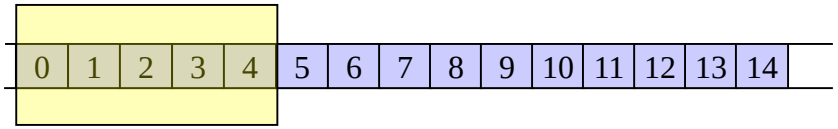
Window size = 4



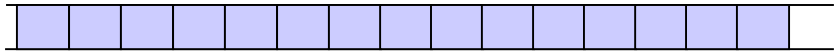
Situation of Variable Window

Step-1

Sender (Host – A)



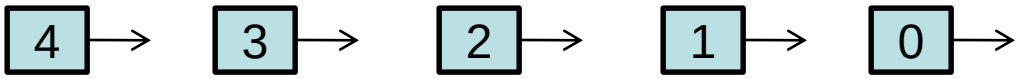
Receiver (Host – B)

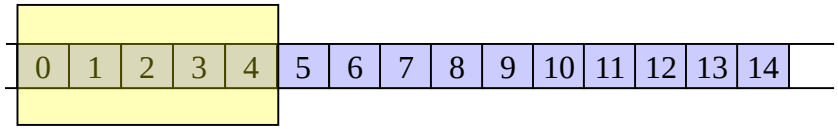


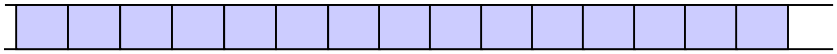
Initialization: A's send window is five

Initialization: B's receive window is five

Step-2

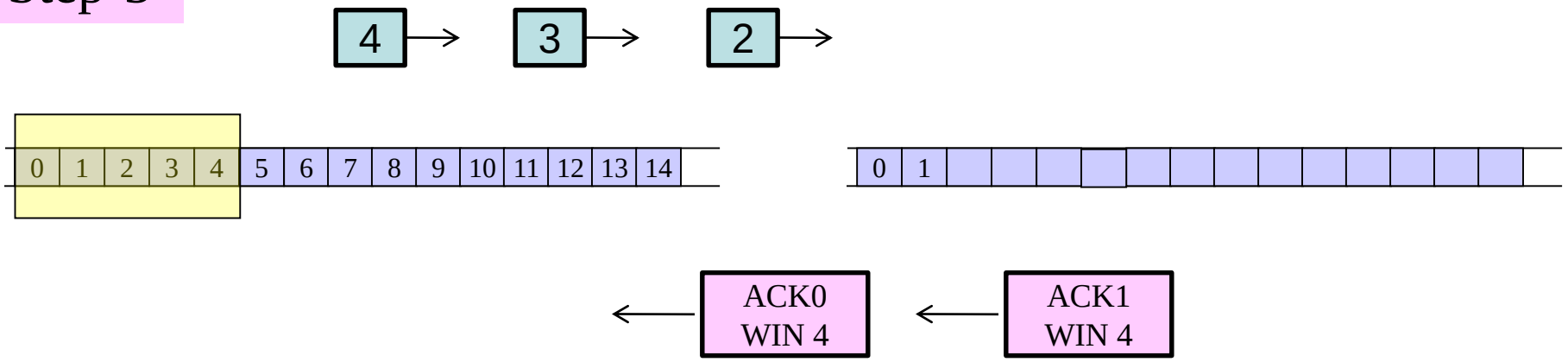






A transmits the first 5 frames

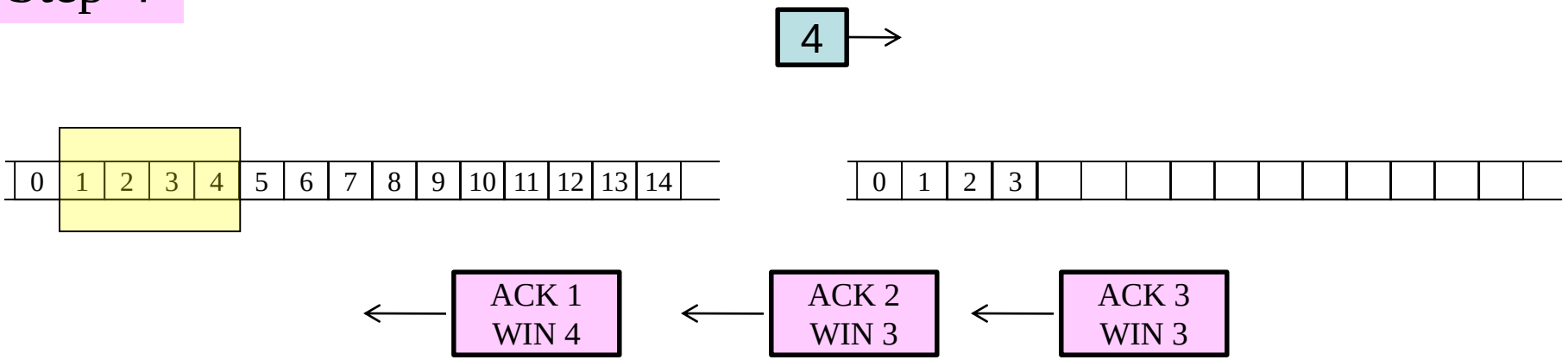
Step-3



A has not received any ACK so it keeps the window size 5 like before

B Receives frame 0 and 1 and sends the corresponding ACK. Each ACK carries a window size of 4. Thus once A receives the ACK for frame 0, it must change its widow size to 4.

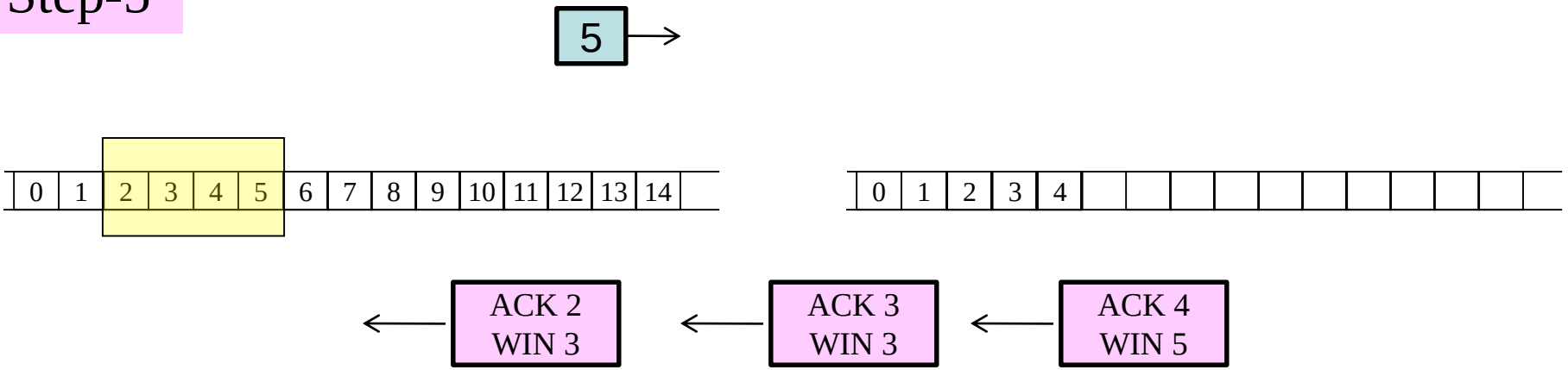
Step-4



A has ACK of frame 0 and advances the left side of its window by one. However, frame 0's ACK had the window size 4 of 4 therefore A cannot advance the right side of its window.

B Receives frame 2 and 3 and sends the corresponding ACK. Each ACK carries a window size of 3. Thus once A receives the ACK, it must change its widow size to 3.

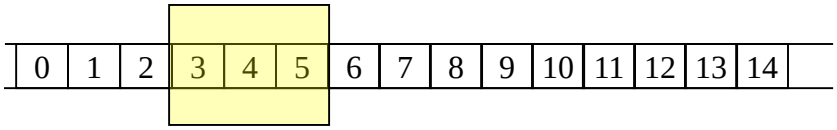
Step-5

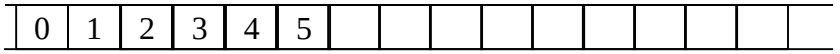


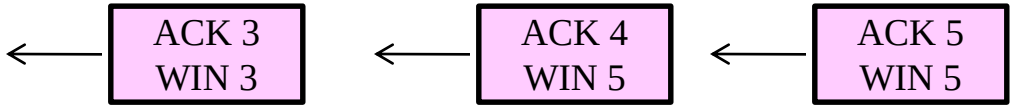
A has ACK of frame 1 and advances the left side of its window by one. However, frame 1's ACK had the window size of 4 therefore A can now advance the right side of its window by one and send frame 5

B Receives frame 4 and sends the corresponding ACK with window size of 5. Thus once A receives the ACK, it must change its widow size to 5.

Step-6



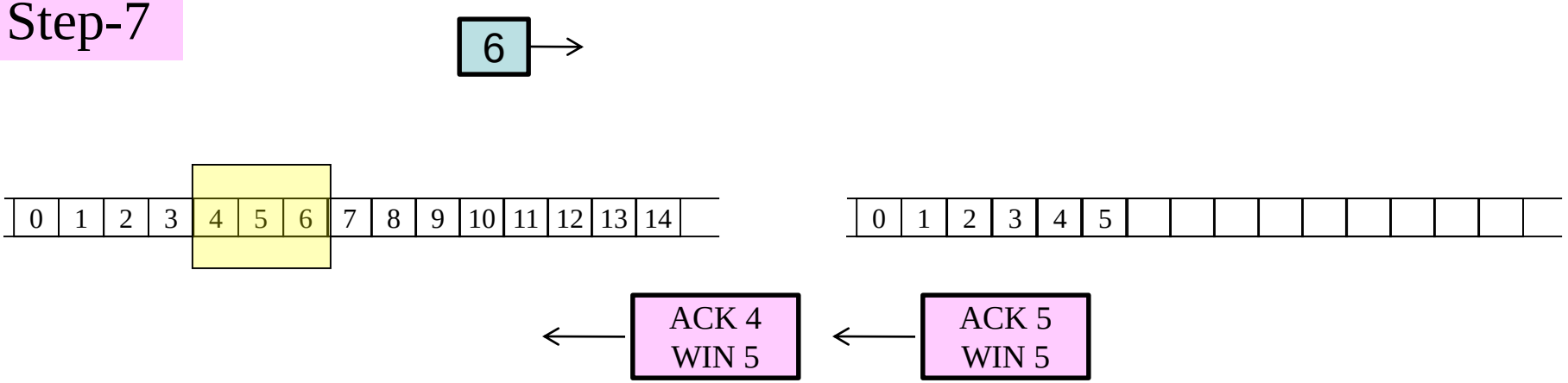




A has ACK of frame 2 and advances the left side of its window by one. However, frame 2's ACK had the window size 3 therefore A cannot advance the right side of its window.

B Receives frame 5 and sends the corresponding ACK with window size of 5.

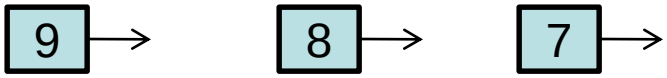
Step-7

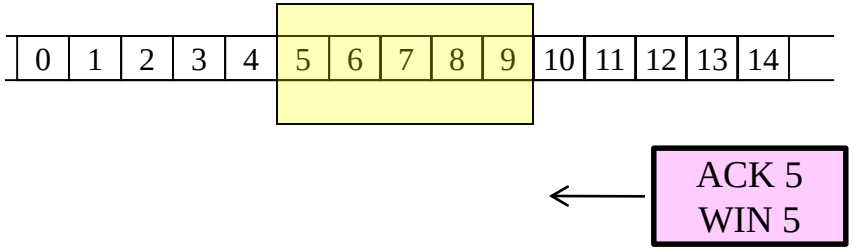


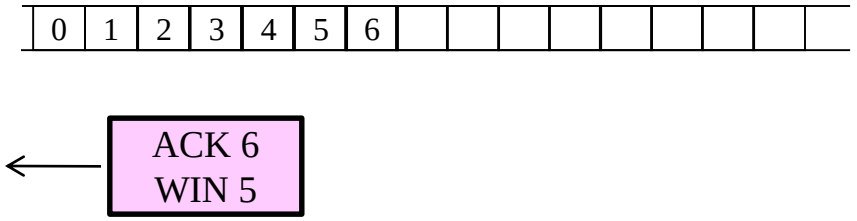
A has ACK of frame 3 and advances both the left and right side of its window by one. This enable the window size of 3 hence A sends frame 6.

B has not received any frame hence sends no ACK .

Step-8



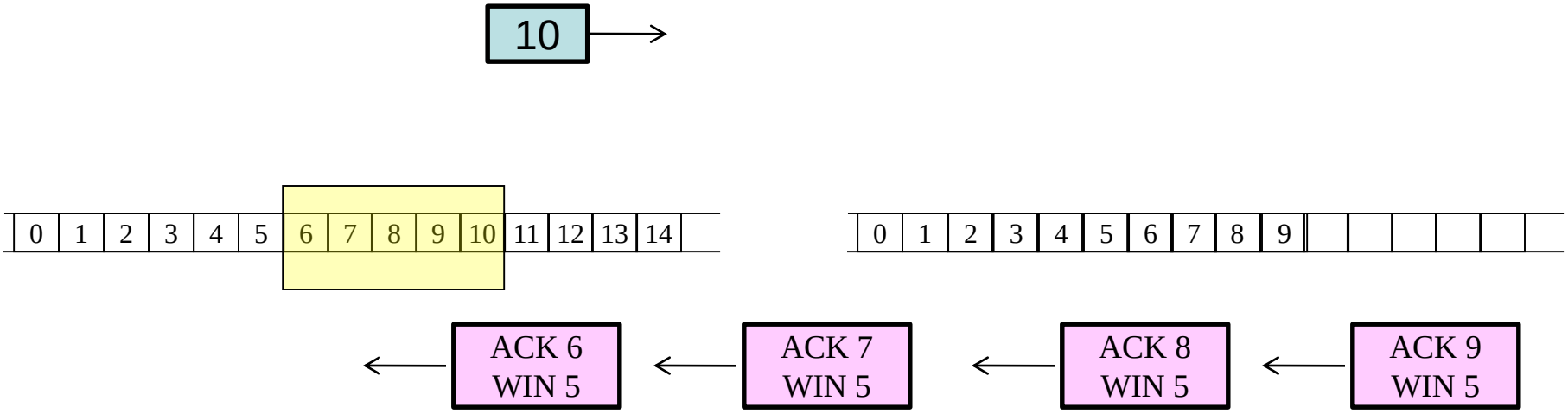




A has ACK of frame 4 and advances the left side by 1 and right side by 3 since ACK-4 provides window size of 5. This permit to send A 3 more frames 7, 8 and 9.

B receives frame 6 and sends ACK with window size of 5.

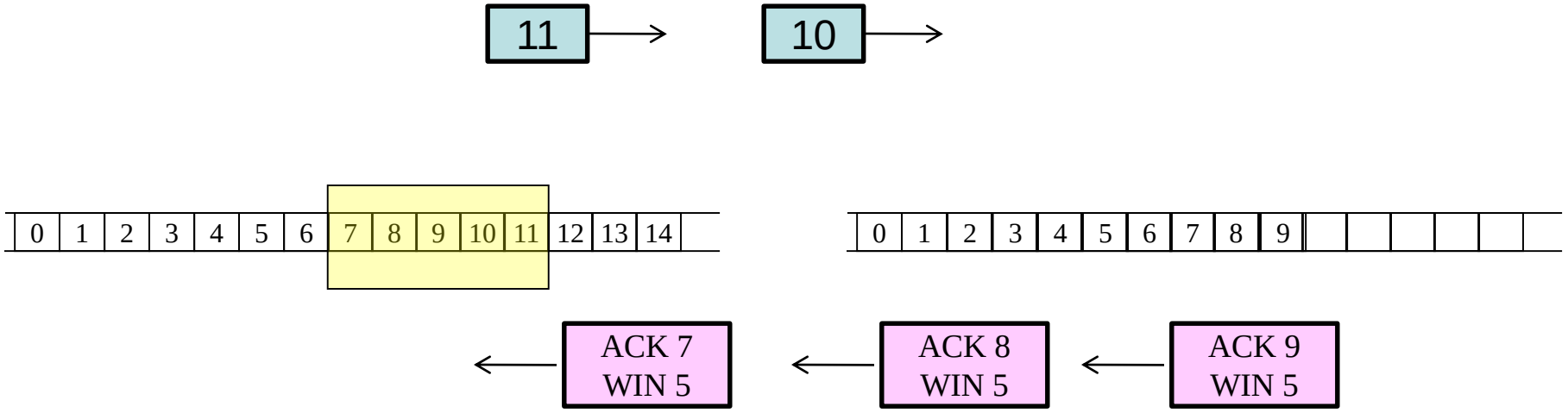
Step-9



A receives ACK of frame 5 which has WIN 5. Thus A advances the left and right side of its window by 1 and transmits frames 10.

B receives frame 7, 8, 9 and sends their corresponding ACK with window size of 5.

Step-10



A receives ACK of frame 6 maintains its window size of 5, and transmits frame 11.

B has not received any frame at this stage and hence sends no ACK.

The Medium Access Control (MAC) Sublayer

In any communication few physical channels (for example each time slot of a TDMA frame is called physical channel) are shared by huge number of users. The MAC sublayer provides a protocol to share limited number channels among a large number of users.

Three LAN protocol at MAC sublayer:

- ✓**Random Access Protocol (**some times called *stochastic* or *statistical*)**, for example ALOHA**
- ✓**Controlled Access Protocol (**token passing **)**
- ✓**CSMA/CD and CSMA/CA**

Random Access Protocol

❑ These protocol employ the philosophy that a node can transmit whenever it has data to transmit. Random access protocols imply contention; in fact, some times we call them contention protocol. Contention is a phenomenon in which more than one entity communicates at the same time.

❑When two or more nodes try to communicate at approximately at the same time, their transmission collide. In LAN terminology we refer to this as a *collision*.

❑For example ALOHA protocol. The efficiency of such protocol is very poor under heavy traffic condition because of frequent collision of packets. Under light traffic condition the throughput of such protocol is high due to low probability of collision. This type of protocol is very simple and can be implemented easily.

Controlled access protocol:

✓To avoid frequent collision of previous protocol a **token** is first secured by a station prior transmission.

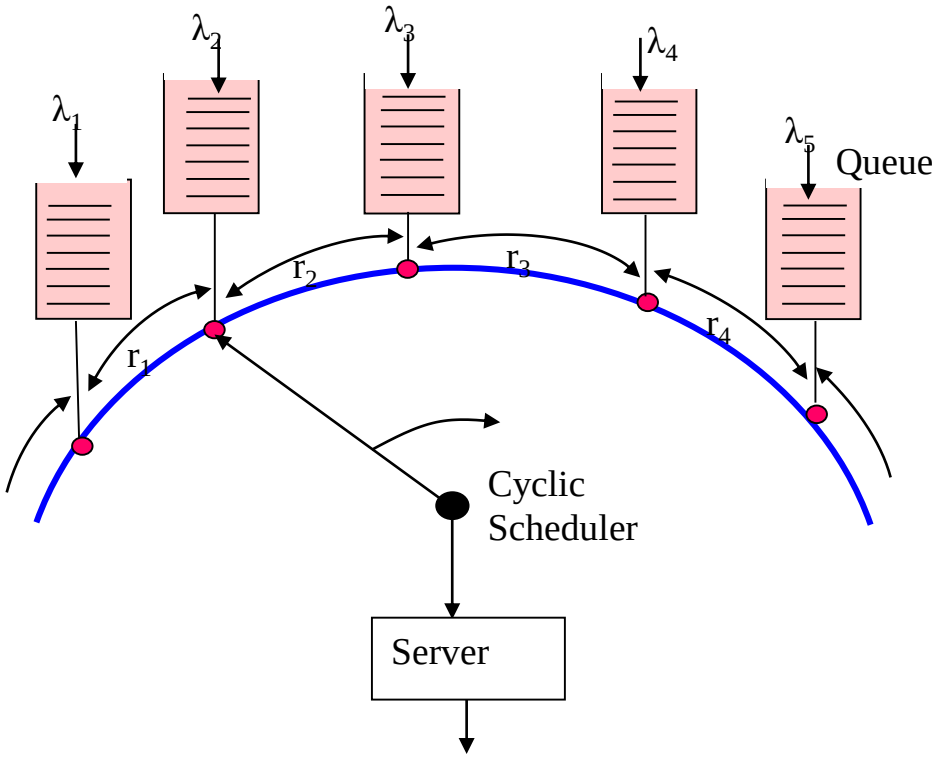
✓Such protocol works efficiently during heavy traffic condition but during low traffic condition a station has to wait unnecessary for its turn. Token ring and token bus network use such protocol.

Example: Token Ring

✓In token ring LAN a special packet called token (permission of access the ring) circulates among a number of terminals in a ring configuration shown in fig. of next slide.

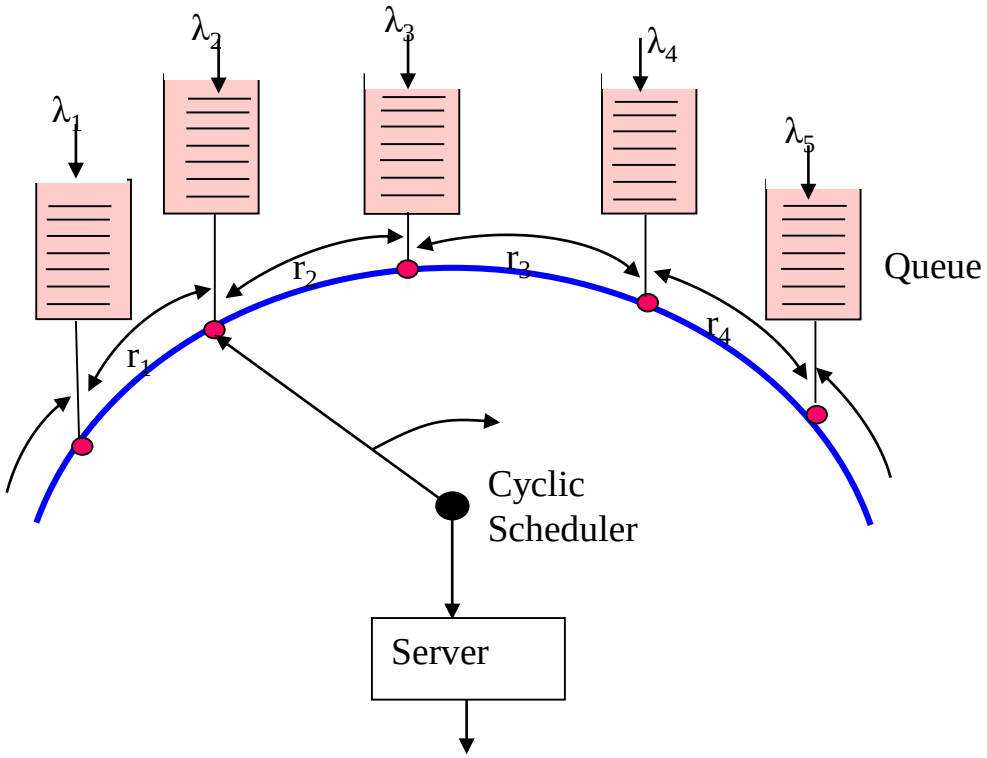
✓Terminals get the opportunity to seize the token sequentially, once the terminal gets the token, the terminal converts it to a busy packet with destination address.

✓After receiving the packet the destination node sends the acknowledgment to the source node.

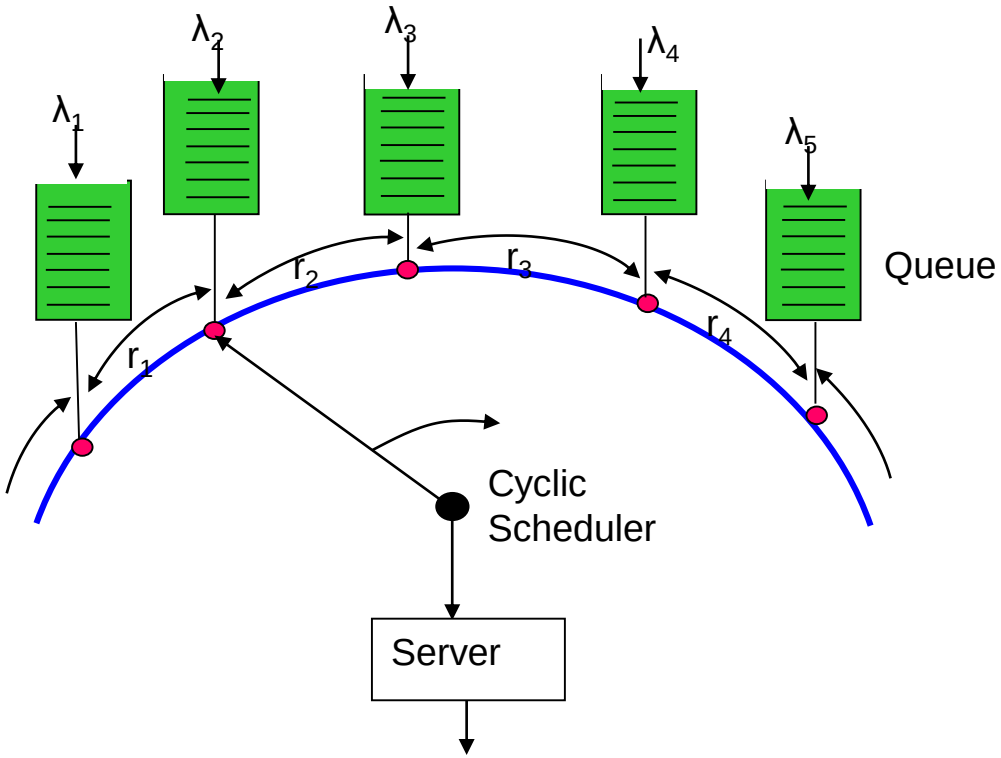


When a station captures a token its serves the queue in one of the following three types of service.

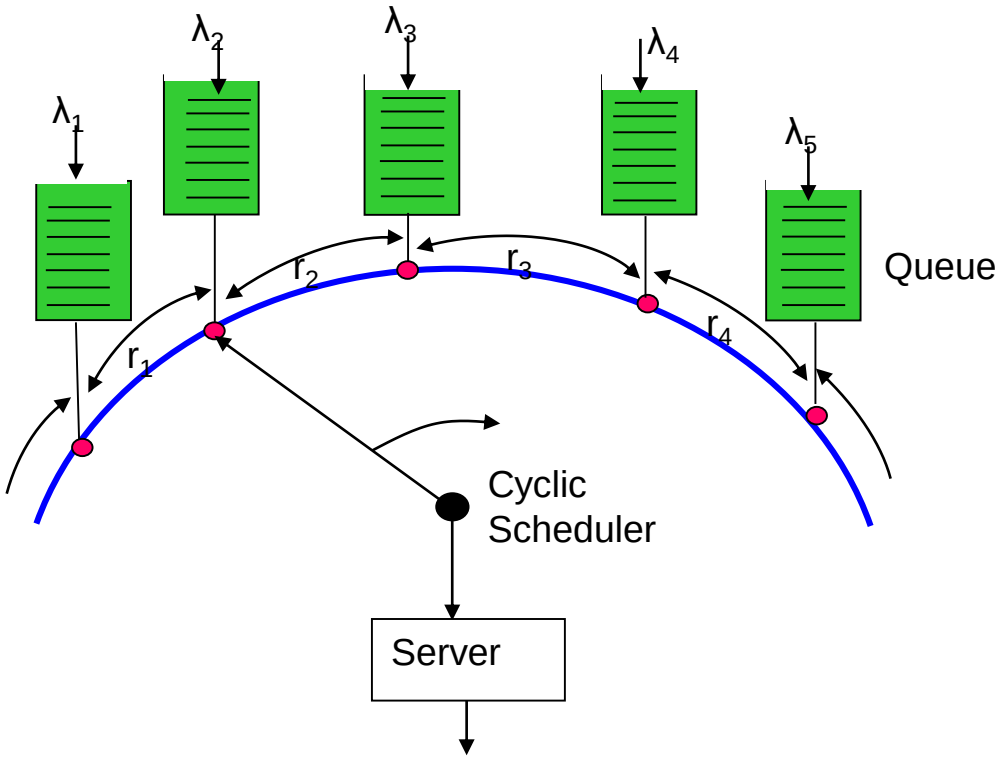
(a) The server serves all the packets (including those received during the transmission) in the queue (no packets of a customer is left) called exhaustive service.



(b) The server serves only its queued packet which arrived just before capturing the token. In other words, the token is made free after transmitting the packets which were waiting before the transmission began i.e. the packets in the queue arrived after getting the token will not be served. This type service is called **gated service**.

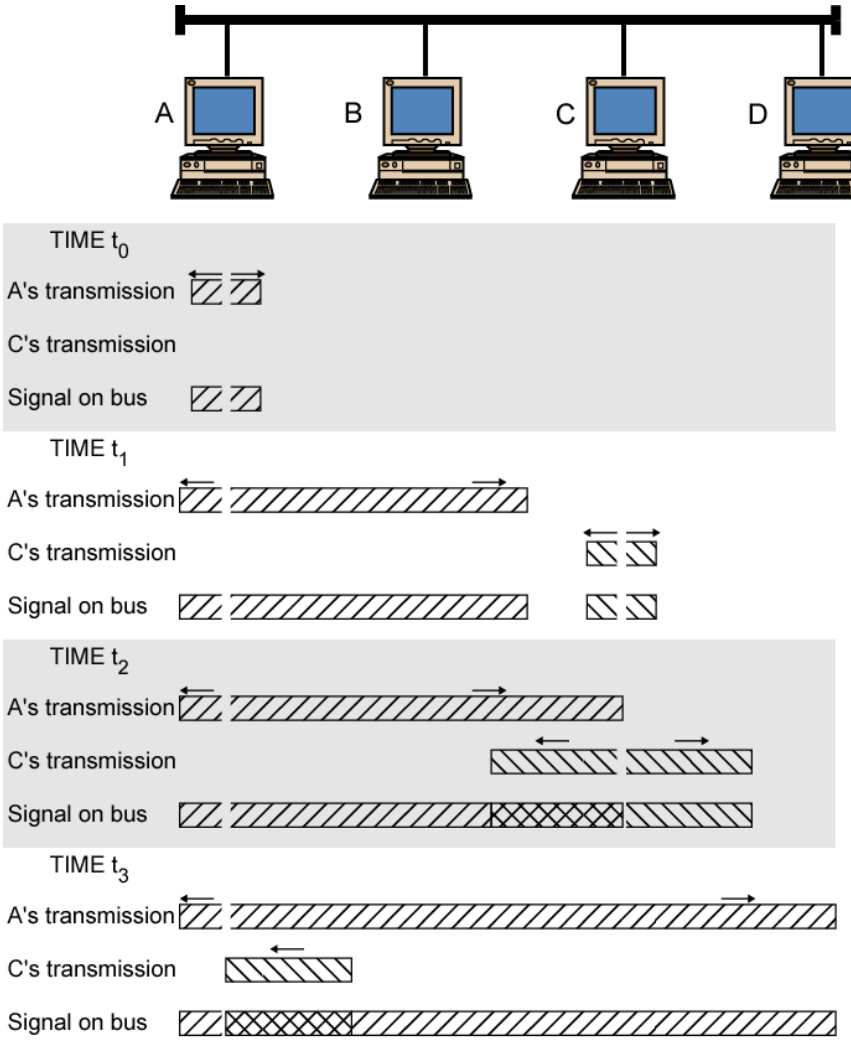


(c) The server serves a limited number of packet *k* may be less than the number of waiting packets in the queue called **limited service**.



Carrier Sense Multiple Access with Collision Detection (CSMA/CD)

- ✓ In carrier sense multiple access with collision detection (CSMA/CD), a node starts its transmission after sensing that there is no carrier in the bus therefore collision of packets from simultaneous users are reduced.
- ✓ If collision occurs nodes can detect it since total signal power will be higher than its transited power.



CSMA/CA

- In wired communication both the transmitted and received signals are equally strong therefore collision can be detected easily, whereas situation is not favorable in wireless link, there collision avoidance (CA) is used in wireless network instead of collision detection (CD).
- Sender usually uses **binary exponential backoff algorithm** under CSMA/CA.